Manual de uso de NovaModelsIA

Guía paso a paso para crear, entrenar, evaluar y usar modelos de IA en tu propio equipo con NovaModelsIA. Todo funciona en local: no hay suscripciones ni APIs de pago.

Índice

1. Qué es y qué puedes hacer
2. Requisitos y primer arranque
3. Conceptos básicos
4. Recorrido por la interfaz
5. Flujo de trabajo común
6. Texto (LLM): ajustar un modelo de lenguaje
7. Visión: clasificar imágenes
8. Audio: ajustar Whisper
9. Imagen: LoRA de Stable Diffusion
10. Modelos: gestionar lo entrenado
11. API local compatible con OpenAI
12. Consejos para 8 GB de VRAM
13. Problemas frecuentes
14. Licencias y responsabilidad

---

1. Qué es y qué puedes hacer

NovaModelsIA tiene cuatro "motores", uno por tipo de modelo:

Pantalla	Qué entrenas	Qué obtienes
Entrenamiento	Un modelo de lenguaje (QLoRA sobre Qwen u otro de Hugging Face)	Un chat que responde con tu estilo o tus datos; exportable a GGUF
Visión	Un clasificador de imágenes (ResNet, MobileNet, EfficientNet)	Un modelo que dice a qué clase pertenece una imagen; exportable a ONNX
Audio	Whisper (tiny, base, small)	Un transcriptor ajustado a tu voz, idioma o vocabulario
Imagen	Un LoRA de Stable Diffusion 1.5	Un estilo o concepto propio al generar imágenes

Además hay una **API local compatible con OpenAI** para usar tus modelos desde otros programas.

---

2. Requisitos y primer arranque

Requisitos: Windows 11 y una GPU NVIDIA con CUDA. Se ha probado con una RTX 3050 de 8 GB.

Abrir la aplicación: si la instalaste, abre `NovaModelsIA` desde el menú Inicio. El programa vive en `%LOCALAPPDATA%\NovaModelsIA` y tus datos (modelos, proyectos, base de datos) en `%LOCALAPPDATA%\NovaModelsIA-data`. Desinstalar o actualizar el programa no borra tus datos.

Para guardar los datos en otro disco (conviene un SSD, porque los modelos pesan decenas de GB), define la variable de entorno `NOVA_DATA` con la carpeta elegida antes de abrir la app.

Indicador de conexión: abajo a la izquierda debe poner **"Backend conectado"**. Si dice "Backend desconectado", cierra la app y ábrela de nuevo.

Instalación de cada motor (solo la primera vez): cada motor vive en su propio entorno aislado. La primera vez que entres en una pantalla (Entrenamiento, Visión, Audio o Imagen) verás un aviso amarillo, "*Motor … no instalado*", con un botón **Instalar motor**. Pulsa el botón y espera: descarga PyTorch con CUDA y otras librerías (varios GB) y necesita internet. El botón muestra "Instalando…" hasta terminar. Después ya no hace falta repetirlo.

Descarga de modelos base: la primera vez que uses un modelo base (por ejemplo `Qwen/Qwen2.5-0.5B-Instruct` o Stable Diffusion 1.5, unos 2 GB) se descarga desde Hugging Face. Esto también necesita internet solo esa vez.

---

3. Conceptos básicos

- **Proyecto**: carpeta lógica que agrupa tus datasets, entrenamientos y modelos. Todo lo demás cuelga de un proyecto, así que crea uno antes de empezar.
- **Dataset**: los datos con los que entrenas. Su formato depende del motor (ver cada sección).
- **Modelo base**: el modelo ya entrenado por otros que vas a ajustar.
- **LoRA / QLoRA**: técnica que no reentrena el modelo entero, sino una capa pequeña añadida (el "adaptador"). Necesita mucha menos memoria. QLoRA además carga el modelo base en 4 bits.
- **Época**: una pasada completa por todos los datos.
- **Loss (pérdida)**: número que mide cuánto se equivoca el modelo; durante un buen entrenamiento baja. Si baja en entrenamiento pero sube en validación, el modelo memoriza en vez de aprender (sobreajuste).
- **Validación**: una parte de los datos que se aparta para medir el modelo con datos que no ha visto.
- **Un solo modelo en GPU a la vez**: con 8 GB de VRAM, cargar otro modelo descarga el anterior, y no se puede cargar nada mientras entrena.

---

4. Recorrido por la interfaz

La barra lateral izquierda tiene:

- El **selector de proyecto** (arriba).
- Los módulos: **Panel, Datasets, Entrenamiento, Evaluación, Inferencia, Visión, Audio, Imagen, Modelos, API local**.
- El indicador de conexión con el backend (abajo).

Módulo	Para qué sirve
Panel	Crear y abrir proyectos; ver CPU, RAM, VRAM y uso de GPU en vivo
Datasets	Importar datasets de texto (JSONL)
Entrenamiento	Entrenar modelos de lenguaje, monitorizar, reanudar y exportar
Evaluación	Medir y comparar modelos de lenguaje con un dataset
Inferencia	Chatear con un modelo de lenguaje (base o entrenado)
Visión	Pestañas *Datos, Entrenar, Evaluar, Probar*
Audio	Pestañas *Datos, Entrenar, Evaluar, Transcribir*
Imagen	Pestañas *Datos, Entrenar, Generar, Comparar*
Modelos	Lista de todo lo entrenado: tamaño, licencia, carpeta y borrado
API local	URL, modelos disponibles y ejemplos de la API

> Consejo: el **Panel** muestra la VRAM en tiempo real. Míralo antes de lanzar un entrenamiento para saber cuánta memoria libre tienes.

---

5. Flujo de trabajo común

Todos los motores siguen la misma secuencia:

1. **Crear un proyecto** (Panel → escribe un nombre → *Crear*).
2. **Instalar el motor** si es la primera vez (botón del aviso amarillo).
3. **Importar un dataset** (te valida y guarda una copia limpia).
4. **Entrenar** (eliges dataset y modelo base, ajustas parámetros o dejas los de por defecto, y pulsas *Iniciar*).
5. **Monitorizar** (progreso, loss, VRAM y registros en vivo).
6. **Evaluar** con datos que el modelo no haya visto.
7. **Probar y usar**: en la propia app o desde otras aplicaciones con la API local.

Un entrenamiento se puede **Detener** en cualquier momento y **Reanudar** después si quedó detenido, falló o se interrumpió (por ejemplo, si se apagó el equipo).

---

6. Texto (LLM): ajustar un modelo de lenguaje

6.1 Preparar el dataset

El archivo debe ser **JSONL**: un registro JSON por línea. Se aceptan estos formatos:

Formato chat (`messages`), el más flexible: admite varios turnos y un mensaje de sistema. El último mensaje debe ser del `assistant`.

```
{"messages": [{"role": "system", "content": "Eres un asistente de soporte técnico amable."}, {"role": "user", "content": "No me enciende el router."}, {"role": "assistant", "content": "Comprueba primero que el cable de corriente esté bien conectado y que la luz de encendido se active. Si no, prueba con otra toma."}]}
```

Otros formatos admitidos (un par pregunta/respuesta por línea):

```
{"instruction": "Resume en una frase: El gato duerme todo el día.", "output": "El gato pasa el día durmiendo."}
{"prompt": "Capital de Perú", "completion": "Lima"}
{"question": "¿Cuántos días tiene una semana?", "answer": "Siete."}
```

(También se acepta el par `input` / `output`.)

Reglas que aplica la validación:

- Los campos no pueden estar vacíos.
- Las líneas que no son JSON válido o no encajan en ningún formato se descartan.
- Los duplicados se eliminan.

Se guarda una **copia limpia** en formato `messages`; tu archivo original no se modifica.

Cuántos ejemplos: para notar un cambio de estilo bastan unos cientos; para enseñar datos o tareas nuevas, lo ideal son miles. Calidad y variedad pesan más que cantidad.

6.2 Importar el dataset

1. Ve a **Datasets**.
2. Pulsa **Examinar...** y elige el `.jsonl` (o escribe la ruta a mano).
3. Escribe un **nombre** y la **licencia del dataset** (por ejemplo, `MIT`; es un dato informativo que queda registrado).
4. Pulsa **Importar y validar**.

Verás un mensaje como: "*Importado: 480 filas válidas (12 inválidas, 8 duplicadas eliminadas)*". Revisa las cifras: si hay muchas inválidas, probablemente el formato no es el esperado.

Con **Vista previa** ves los primeros ejemplos ya normalizados.

6.3 Entrenar

Ve a **Entrenamiento**. Si aparece el aviso amarillo, instala el **motor LLM**.

En *Nuevo entrenamiento (LoRA / QLoRA)* elige:

Campo	Qué es	Valor por defecto
Dataset	El que importaste	el primero
Modelo base (Hugging Face)	Identificador del repositorio. Con **Licencia** consultas su licencia	`Qwen/Qwen2.5-0.5B-Instruct`
Épocas	Pasadas por los datos (admite 0,5)	1
Tamaño de lote	Ejemplos por paso	1
Acumulación de gradiente	Pasos que se acumulan antes de actualizar; lote efectivo = lote × acumulación	8
Learning rate	Velocidad de aprendizaje	0,0002
Longitud máx. de secuencia	Tokens máximos por ejemplo; más longitud, más VRAM	512
LoRA rank (r)	Capacidad del adaptador	16
LoRA alpha	Escala del adaptador (suele ser 2 × r)	32
Guardar cada N pasos	Frecuencia de los puntos de guardado	50
Optimizador	`paged_adamw_8bit` ahorra memoria	`paged_adamw_8bit`
Scheduler	Cómo varía el learning rate	`cosine`
Cuantización 4-bit (QLoRA)	Reduce mucho la VRAM	activada
Gradient checkpointing	Ahorra VRAM a costa de algo de velocidad	activado

Valores iniciales recomendados para 8 GB: deja los de por defecto. Es una configuración conservadora que cabe sin problemas con un modelo de 0,5B. Para modelos de 1,5B a 3B mantén lote 1, 4-bit y gradient checkpointing, y baja la longitud de secuencia si te quedas sin memoria.

Pulsa **Iniciar entrenamiento**. El botón está desactivado si no hay motor instalado o no hay dataset.

6.4 Monitorizar

En la lista *Entrenamientos* pulsa un trabajo para abrir el **Monitoreo**:

- **Paso** actual / total, **Loss**, **Eval loss** (si hay validación) y **VRAM pico**.
- La **gráfica de pérdida**.
- Los **registros** del proceso (los últimos 200 renglones).

Estados posibles: *En cola, Entrenando, Completado, Falló, Detenido, Interrumpido*.

Botones de cada trabajo:

- **Detener**: mientras está en cola o entrenando.
- **Reanudar**: si está detenido, falló o se interrumpió. Continúa desde el último punto guardado.
- **Exportar fusionado**: cuando está completado. Fusiona el adaptador con el modelo base en un modelo independiente.
- **Exportar GGUF**: cuando está completado. Genera un archivo GGUF para usarlo en programas como llama.cpp.

6.5 Probar el modelo (Inferencia)

1. Ve a **Inferencia**.
2. Elige *Modelo base (sin adaptador)* o un **Entrenamiento #N** completado.
3. Si eliges el modelo base, puedes escribir el identificador de Hugging Face.
4. Pulsa **Cargar**, espera y escribe en el chat. Con el control de **Temperatura** controlas la creatividad (0 = más determinista, 1,5 = más variado).
5. **Descargar** libera la VRAM cuando termines.

Truco para comprobar que tu entrenamiento sirvió: haz la **misma pregunta** al modelo base y a tu entrenamiento, y compara.

6.6 Evaluar (Evaluación)

La evaluación mide la pérdida y la **perplejidad** sobre la respuesta esperada, y compara la respuesta generada con la de referencia (**F1 de palabras** y **coincidencia exacta**).

1. Ve a **Evaluación**.
2. Elige un dataset **que no hayas usado para entrenar**. Si usas el mismo, la app avisa de que el resultado estará inflado.
3. Marca los modelos a comparar (puedes incluir el modelo base como referencia).
4. Pulsa **Evaluar N modelos**.

En **Comparación** el mejor resultado de cada métrica sale en verde. Pulsa una fila para ver ejemplos concretos de prompt, respuesta esperada y respuesta del modelo.

6.7 Ejemplo completo: un asistente con tu estilo

1. Crea el proyecto `Soporte`.
2. Prepara `soporte_train.jsonl` con 300 preguntas y respuestas en formato `messages`, y `soporte_test.jsonl` con otras 50 distintas.
3. En **Datasets**, importa ambos (`soporte_train` y `soporte_test`).
4. En **Entrenamiento**, elige `soporte_train`, deja los valores por defecto y sube las épocas a 3. Inicia el entrenamiento.
5. Vigila que el **loss** baje. Cuando termine (*Completado*), ve a **Inferencia**, carga ese entrenamiento y pregúntale algo del dominio.
6. En **Evaluación**, evalúa el modelo base y tu entrenamiento sobre `soporte_test` y compara.
7. Si te gusta, **Exportar GGUF** para usarlo fuera, o sírvelo con la API local (sección 11).

---

7. Visión: clasificar imágenes

Un clasificador aprende a asignar una **etiqueta** (clase) a cada imagen.

7.1 Preparar el dataset

Una carpeta con **una subcarpeta por clase**; el nombre de la subcarpeta es la etiqueta:

```
mis_imagenes/
  gato/
    001.jpg
    002.jpg
  perro/
    001.jpg
    002.png
  pajaro/
    ...
```

Formatos admitidos: `.jpg .jpeg .png .bmp .webp`. La app comprueba cada imagen y avisa de:

- Menos de **2 clases**.
- Clases con **menos de 10 imágenes**.
- Clases **muy desbalanceadas** (más de 5 a 1 entre la mayor y la menor).

Como guía: 50–100 imágenes por clase ya dan buenos resultados con *fine-tuning*; con 10 funcionará pero será frágil.

7.2 Pestaña Datos

1. Ve a **Visión** → **Datos** (instala el motor si es la primera vez).
2. **Examinar…** y elige la carpeta raíz (`mis_imagenes`).

3. Pon un nombre y la licencia, y pulsa **Importar**.

Las imágenes válidas se copian al proyecto. Los avisos aparecen en la lista *Datasets de imágenes*.

7.3 Pestaña Entrenar

Campo	Qué es	Valor por defecto
Dataset	El que importaste	—
Modelo	Arquitectura (ver tabla)	ResNet-18
Épocas	Pasadas por los datos	10
Tamaño de lote	Imágenes por paso	32
Learning rate	Velocidad de aprendizaje	0,001
Tamaño de imagen (px)	Resolución de entrada	160
Fracción de validación	Parte de los datos reservada para medir	0,2
Congelar el backbone	Solo entrena la última capa; más rápido, útil con pocos datos	desactivado
Aumento de datos	Giros y variaciones para generalizar mejor	activado

Modelos disponibles:

Modelo	Parámetros	Cuándo usarlo
ResNet-18	11,7 M	Rápido y ligero; buen punto de partida
MobileNetV3 Large	5,5 M	Muy ligero, ideal para CPU
EfficientNet-B0	5,3 M	Buen equilibrio entre precisión y tamaño
ResNet-50	25,6 M	Más preciso, más lento

Pulsa **Iniciar** y sigue el monitoreo (pérdida, precisión de validación). Un trabajo completado se puede **Exportar ONNX** para usarlo en otras aplicaciones.

7.4 Pestaña Evaluar

Elige un modelo entrenado y un dataset **con imágenes que no se usaron para entrenar** (importa otro con las mismas clases). Se calculan métricas y se muestra la **matriz de confusión** (qué clases se confunden entre sí) y las métricas por clase. Pulsa una fila de resultados para verlas.

7.5 Pestaña Probar

Elige un clasificador entrenado, pulsa **Cargar**, sube una imagen y verás la clase predicha. **Descargar** libera la VRAM.

7.6 Ejemplo completo: distinguir tornillos de tuercas

1. Fotografía 60 tornillos y 60 tuercas y organízalos en `piezas/tornillo/` y `piezas/tuerca/`. Guarda aparte 15 fotos de cada una en `piezas_test/` con la misma estructura.
2. **Visión** → **Datos**: importa `piezas` y `piezas_test` como datasets distintos.
3. **Entrenar**: dataset `piezas`, ResNet-18, valores por defecto. Inicia.
4. **Evaluar**: modelo entrenado sobre `piezas_test`. Revisa la matriz de confusión.
5. **Probar**: carga el modelo y sube una foto nueva.

---

8. Audio: ajustar Whisper

Sirve para que Whisper transcriba mejor tu voz, un acento, un idioma o vocabulario técnico.

8.1 Preparar el dataset

Una carpeta con los clips de audio y un archivo de transcripciones:

```
mis_audios/
  clip001.wav
  clip002.wav
  metadata.csv
```

`metadata.csv` con las columnas `file,text`:

```
file,text
clip001.wav,hola buenos días quiero consultar mi saldo
clip002.wav,necesito reiniciar la contraseña de mi cuenta
```

También se acepta `metadata.jsonl` con `file` (o `file_name`) y `text` (o `transcription`).

Requisitos de los clips:

- Formatos `.wav .flac .ogg .mp3`.
- Duración entre **0,3 y 30 segundos**.
- La transcripción debe reflejar exactamente lo que se dice.

Con unas decenas de minutos de audio bien transcrito ya se nota el ajuste.

8.2 Pestaña Datos

Audio → **Datos** (instala el motor si hace falta) → **Examinar…** → nombre y licencia → **Importar**. Se valida cada clip y se avisa de los problemas.

8.3 Pestaña Entrenar

Campo	Qué es	Valor por defecto
Dataset	El importado	—
Modelo base	Whisper tiny / base / small	`openai/whisper-tiny`
Idioma	Idioma de las transcripciones	Español
Épocas	Pasadas por los datos	5
Tamaño de lote	Clips por paso	8
Acumulación de gradiente	—	1
Learning rate	Mantenlo bajo en Whisper	0,00001
Fracción de validación	Parte para medir	0,15
Congelar encoder	Entrena solo el decodificador; más ligero	desactivado
Gradient checkpointing	Ahorra VRAM	desactivado

Modelos: **tiny** (39 M, muy rápido, precisión limitada), **base** (74 M, buen punto de partida), **small** (244 M, más preciso y ajustado para 8 GB). Empieza con *tiny* o *base* para validar tu flujo y pasa a *small* cuando todo funcione.

8.4 Pestaña Evaluar

Mide **WER** (tasa de error de palabras) y **CER** (de caracteres) sobre texto normalizado, sin mayúsculas ni puntuación. **Menos es mejor.** Usa clips que no hayan intervenido en el entrenamiento (si no, la app avisa de que el WER estará subestimado). Puedes comparar el modelo base con tus entrenamientos.

8.5 Pestaña Transcribir

1. Elige *Modelo base* o un entrenamiento, y el idioma (o *Detectar idioma*).
2. Pulsa **Cargar**.
3. Elige un archivo de audio (la app no graba; usa un archivo ya existente) y obtienes el texto.

Los audios de más de 28 s se cortan en ventanas, por lo que alguna palabra puede partirse en el corte.

8.6 Ejemplo completo: transcribir llamadas con vocabulario propio

1. Graba 100 frases cortas con los términos de tu negocio y escribe `metadata.csv`. Reserva 20 en otra carpeta para evaluar.
2. **Audio** → **Datos**: importa ambas carpetas.
3. **Entrenar**: `openai/whisper-base`, español, valores por defecto.
4. **Evaluar**: compara *Whisper base* contra tu entrenamiento sobre las 20 frases reservadas; el WER del tuyo debería ser menor.
5. **Transcribir**: prueba con un audio real.

---

9. Imagen: LoRA de Stable Diffusion

Enseña a Stable Diffusion 1.5 un estilo o un concepto propio. Solo se admite SD 1.5, porque SDXL no cabe en 8 GB para entrenar. La primera vez descarga unos 2 GB.

> **Aviso**: Stable Diffusion se usa **sin comprobador de seguridad**; las imágenes generadas no se filtran. La licencia del modelo (CreativeML OpenRAIL-M) tiene restricciones de uso que debes respetar.

9.1 Preparar el dataset

Una carpeta con imágenes (`.png .jpg .webp`), cada una con una descripción, de una de estas formas (por orden de prioridad):

1. Un `metadata.csv` con columnas `file,text`.
2. Un `.txt` **con el mismo nombre** que la imagen (`foto1.png` → `foto1.txt`).
3. Una **descripción por defecto** para todas las imágenes (útil para un estilo uniforme).

Incluye una **palabra clave propia** inventada en las descripciones, por ejemplo `novastyle`. Luego bastará escribirla en el prompt para invocar lo aprendido. Las imágenes sin descripción se rechazan.

Orientación: 15–30 imágenes variadas de un mismo estilo o sujeto ya dan resultados. Más variedad en composiciones mejora la generalización.

9.2 Pestaña Datos

Imagen → **Datos** → **Examinar…** → nombre, descripción por defecto (opcional) y **licencia de las imágenes** (¿tienes derecho a entrenar con ellas?) → **Importar**.

9.3 Pestaña Entrenar

Campo	Qué es	Valor por defecto
Dataset	El importado	—
Resolución	Píxeles de entrenamiento	512
Pasos de entrenamiento	Duración total	300
Tamaño de lote	—	1
Acumulación de gradiente	—	4
Learning rate	—	0,0001
Rank del LoRA	Capacidad del adaptador	8
Alpha del LoRA	Escala	8
Imagen de muestra cada N pasos	Genera una imagen de control (0 = nunca)	100
Prompt de validación	Texto para esas imágenes de muestra	vacío
Gradient checkpointing	Ahorra VRAM	activado

Escribe un **prompt de validación** que incluya tu palabra clave (por ejemplo `a cat, novastyle`): así ves cómo evoluciona el aprendizaje a lo largo del entrenamiento.

9.4 Pestaña Generar

1. Elige *Solo el modelo base* o uno de tus LoRA entrenados y pulsa **Cargar**.
2. Escribe el **prompt**, en inglés (funciona mejor con SD 1.5), con tu palabra clave. Opcionalmente, un *prompt negativo* con lo que quieres evitar.
3. Ajusta pasos, guidance, semilla, ancho, alto y número de imágenes (128–768 px en la API; en esta pantalla puedes variar el tamaño).
4. Pulsa **Generar**.

Fijar la **semilla** permite repetir el mismo resultado y comparar de forma justa.

9.5 Pestaña Comparar

Genera con el **mismo prompt y la misma semilla** usando el modelo base y cada LoRA marcado. Escribe un prompt por línea (hasta 8) y semillas separadas por comas (hasta 4). Incluye al menos un prompt con combinaciones que no aparecían en el entrenamiento, para comprobar si aprendió el concepto o solo memorizó las imágenes.

9.6 Ejemplo completo: un estilo de ilustración

1. Reúne 20 ilustraciones propias en `estilo/`. Pon como descripción por defecto: `a flat illustration, novastyle`.
2. **Imagen** → **Datos**: importa la carpeta con tu licencia.

3. **Entrenar**: valores por defecto, prompt de validación `a red circle, novastyle`.
4. Cuando termine, **Generar** con tu LoRA y el prompt `a house on a hill, novastyle`.
5. **Comparar**: mismo prompt con *base* y tu LoRA, para ver qué aporta.

---

10. Modelos: gestionar lo entrenado

La pantalla **Modelos** lista los entrenamientos de LLM y de visión del proyecto:

- Identificador, modelo base, motor, estado, fecha y tamaño.
- El *loss* final o la precisión de validación.
- **Licencia**: consulta la del modelo base en Hugging Face. El modelo resultante hereda sus condiciones.
- **Abrir carpeta**: abre su carpeta en el Explorador (solo en la app de escritorio).
- **Borrar**: pide confirmación (*Confirmar borrado*) y no está disponible mientras el trabajo está en cola o en curso.
- Los archivos generados (modelo fusionado, GGUF, ONNX) con su tamaño.

Arriba verás el espacio total en disco. Revisa esta pantalla de vez en cuando: los modelos ocupan mucho.

---

11. API local compatible con OpenAI

Sirve tus modelos a otros programas en `http://127.0.0.1:8765/v1`, solo accesible desde tu equipo. La pantalla **API local** muestra la URL (con botón *Copiar URL*), el modelo cargado ahora (con *Descargar*), los modelos disponibles y ejemplos listos para copiar.

Reglas importantes:

- Solo sirve modelos **ya descargados o entrenados**; nunca descarga nada por su cuenta.
- Hay **un solo modelo en GPU**: pedir otro descarga el actual.
- Si hay un **entrenamiento en curso**, responde con error 503 (`gpu_busy`).
- Para exigir clave, define `NOVA_API_KEY` antes de arrancar y envía `Authorization: Bearer <clave>`.

11.1 Chat

```
curl http://127.0.0.1:8765/v1/chat/completions -H "Content-Type: application/json" -d '{
  "model": "nova-llm-16",
  "messages": [{"role": "user", "content": "Hola"}],
  "max_tokens": 100
}'
```

El nombre del modelo (`nova-llm-16`) es solo un ejemplo: copia el que aparezca en *Modelos disponibles*.

Con el SDK de OpenAI de Python y *streaming*:

```
from openai import OpenAI
client = OpenAI(base_url="http://127.0.0.1:8765/v1", api_key="no-se-usa")

for chunk in client.chat.completions.create(
        model="nova-llm-16", stream=True,
        messages=[{"role": "user", "content": "Hola"}]):
    print(chunk.choices[0].delta.content or "", end="")
```

11.2 Transcripción de audio

```
with open("audio.wav", "rb") as f:   # wav, flac, ogg o mp3 (máx. 25 MB)
    r = client.audio.transcriptions.create(model="<modelo-de-audio>", file=f,
language="es")
print(r.text)
```

11.3 Generación de imágenes

```
import base64
r = client.images.generate(
    model="<modelo-de-imagen>", prompt="a flat illustration of a red circle, novastyle",
    size="512x512", response_format="b64_json",
    extra_body={"seed": 1, "num_inference_steps": 25})
open("imagen.png", "wb").write(base64.b64decode(r.data[0].b64_json))
```

11.4 Límites

- **Chat**: roles system, user y assistant; solo texto; `temperature`, `top_p`, `max_tokens` y `stream`. No hay `n>1`, `stop`, herramientas ni imágenes de entrada. El recuento de tokens es aproximado.
- **Audio**: solo formatos de respuesta `json` y `text`; sin marcas de tiempo.
- **Imágenes**: solo `b64_json` (no se sirven URLs), de 128 a 768 px.

---

12. Consejos para 8 GB de VRAM

- **Cierra otras aplicaciones que usen la GPU** (juegos, navegadores con aceleración, otros modelos) antes de entrenar. Compruébalo en el Panel.

- **LLM**: modelos de 0,5B a 3B con QLoRA. Mantén lote 1, 4-bit y gradient checkpointing. Si aparece un error de memoria, baja la longitud máxima de secuencia (por ejemplo, de 512 a 256) o el rank del LoRA.
- **Audio**: Whisper tiny, base o small. Si *small* se queda sin memoria, activa *Gradient checkpointing*, baja el lote y sube la acumulación.
- **Imagen**: entrena a 512 px; deja activado el gradient checkpointing.
- **Visión**: si falta memoria, baja el lote o el tamaño de imagen.
- **Prueba primero en pequeño**: un entrenamiento corto (pocos pasos o 1 época) confirma que datos y configuración funcionan antes de dedicar horas.
- **Guarda en SSD**: los modelos cargan mucho más rápido.

---

13. Problemas frecuentes

Síntoma	Qué hacer
"Backend desconectado"	Cierra y vuelve a abrir la aplicación
El botón de entrenar está desactivado	Falta instalar el motor o importar un dataset
La instalación del motor falla	Revisa tu conexión a internet y pulsa de nuevo el botón; el mensaje de error aparece bajo el botón
Error de memoria (*CUDA out of memory*)	Aplica los consejos de la sección 12 y reanuda o relanza
El entrenamiento se interrumpió (apagón, cierre)	Pulsa **Reanudar** en ese trabajo
La API responde 503 `gpu_busy`	Hay un entrenamiento en curso; espera a que termine
"Importado: … inválidas" muy altas	Revisa el formato del archivo (sección 6.1, 7.1, 8.1 o 9.1)
Las métricas de evaluación parecen demasiado buenas	Estás evaluando con datos usados en el entrenamiento; usa un dataset aparte
Los modelos ocupan demasiado	Bórralos desde **Modelos** o mueve los datos a otro disco con `NOVA_DATA`
El selector de archivos o *Abrir carpeta* no funciona	Solo funcionan en la app de escritorio; en otros contextos escribe la ruta a mano

---

14. Licencias y responsabilidad

- Cada modelo base tiene su propia licencia. La app permite consultarla (botón **Licencia**); revísala antes de usar el modelo de forma comercial. Por ejemplo, Qwen2.5-3B tiene una licencia propia distinta de la Apache-2.0 de los modelos de 0,5B, 1,5B y 7B.
- Stable Diffusion 1.5 usa CreativeML OpenRAIL-M, con restricciones de uso.

- Los modelos entrenados **heredan** las condiciones del modelo base.
- Comprueba la licencia de tus datasets y de tus imágenes y audios: entrenar con ellos es responsabilidad tuya.